Predicting Vehicle Fuel Efficiency Using Machine Learning

Isaiah Alvarado-Ramirez

ITCS 3156 – Machine Learning

Professor Minwoo Lee

December 5, 2025

Introduction

Fuel efficiency is one of the most important performance measures in automotive engineering, environmental science, and consumer decision-making. A vehicle's miles per gallon (MPG) depends on factors such as engine size, vehicle weight, horsepower, and aerodynamics. Predicting MPG with machine learning is valuable because it can help manufacturers build more efficient cars, support environmental research, and help consumers better understand vehicle performance.

In this project, I use a publicly available automobile dataset from Kaggle to build two machine learning models, Ordinary Least Squares (OLS) and Least Mean Squares (LMS) gradient descent, to predict MPG from several car features. OLS provides a closed form analytical solution, while LMS performs iterative optimization. Comparing both methods allows me to evaluate the impact of preprocessing, feature scaling, and algorithm choice on predictive accuracy.

This problem matters because fuel efficiency is directly connected to sustainability and reducing emissions. Some challenges I encountered include handling missing data, choosing appropriate preprocessing steps, and tuning the LMS learning rate for stable convergence. Linear regression is a reasonable starting point for this type of prediction task because it is interpretable and efficient.

Data

The dataset used is the *Auto MPG Dataset* from Kaggle. It includes over 10,000 samples and the following features:

- cylinders
- displacement
- horsepower
- weight
- acceleration
- model year
- origin

The target variable is MPG.

Data Exploration & Visualization

A histogram of MPG reveals that most vehicles fall between 15–30 MPG:

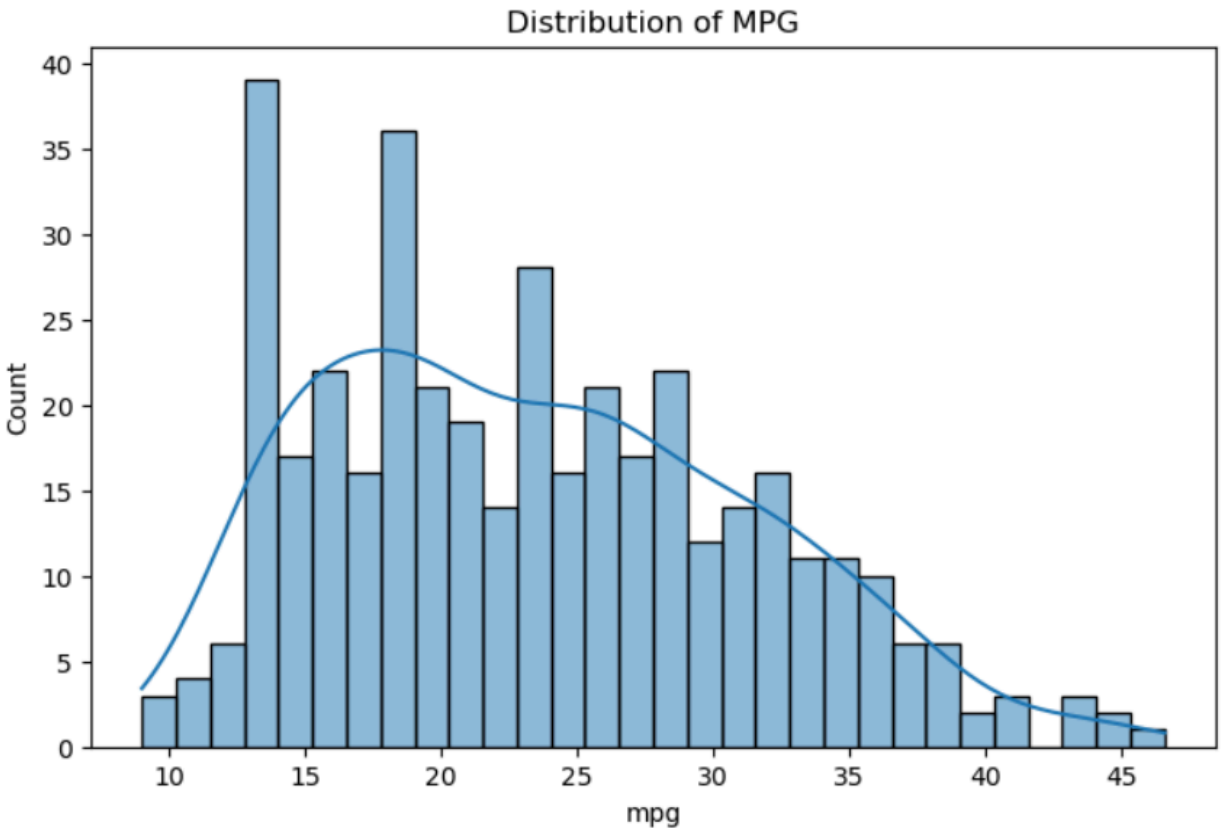

Figure 1. Distribution of MPG

Shows the frequency distribution of miles-per-gallon values in the dataset. Most vehicles fall between 15–30 MPG, with fewer high-efficiency cars.

A scatterplot of horsepower vs. MPG shows a strong negative correlation:

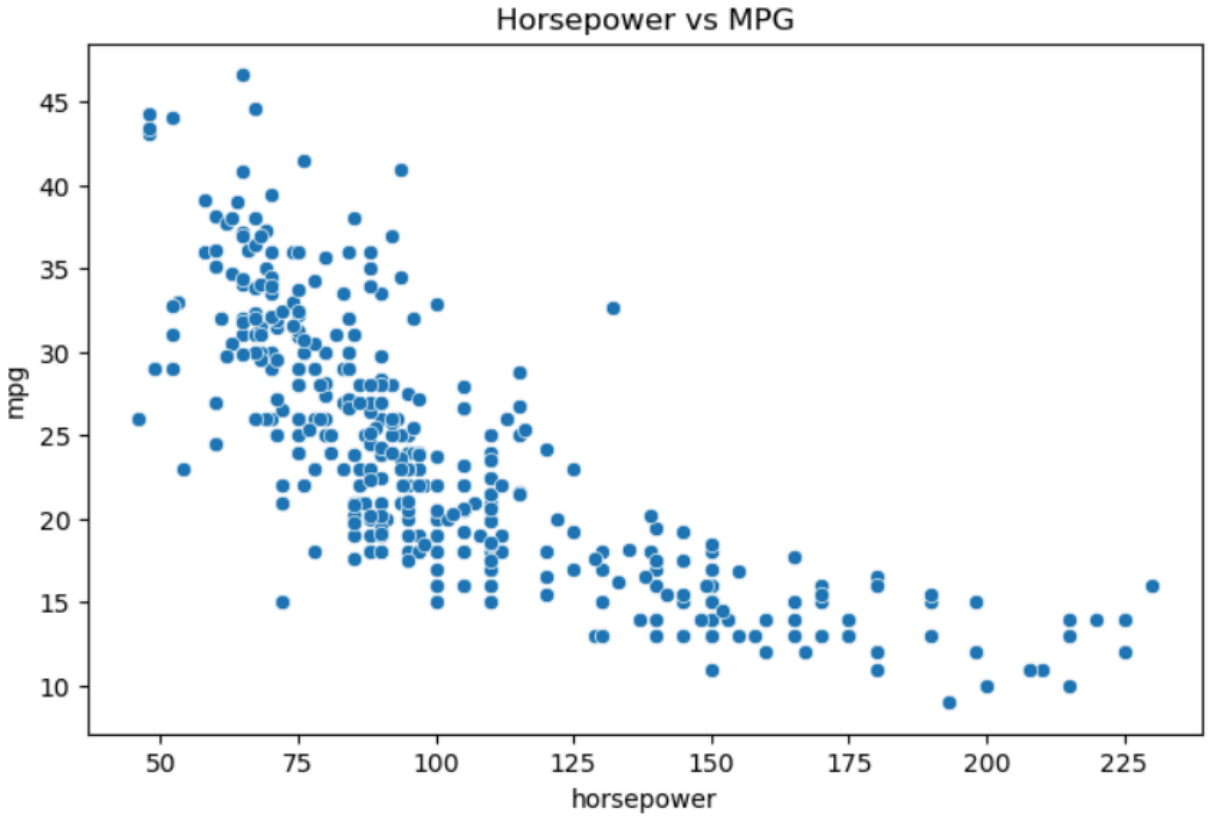

Figure 2. Horsepower vs. MPG

A scatterplot showing a strong negative correlation between horsepower and fuel efficiency. Higher power generally results in lower MPG.

Likewise, heavier vehicles generally achieve lower MPG:

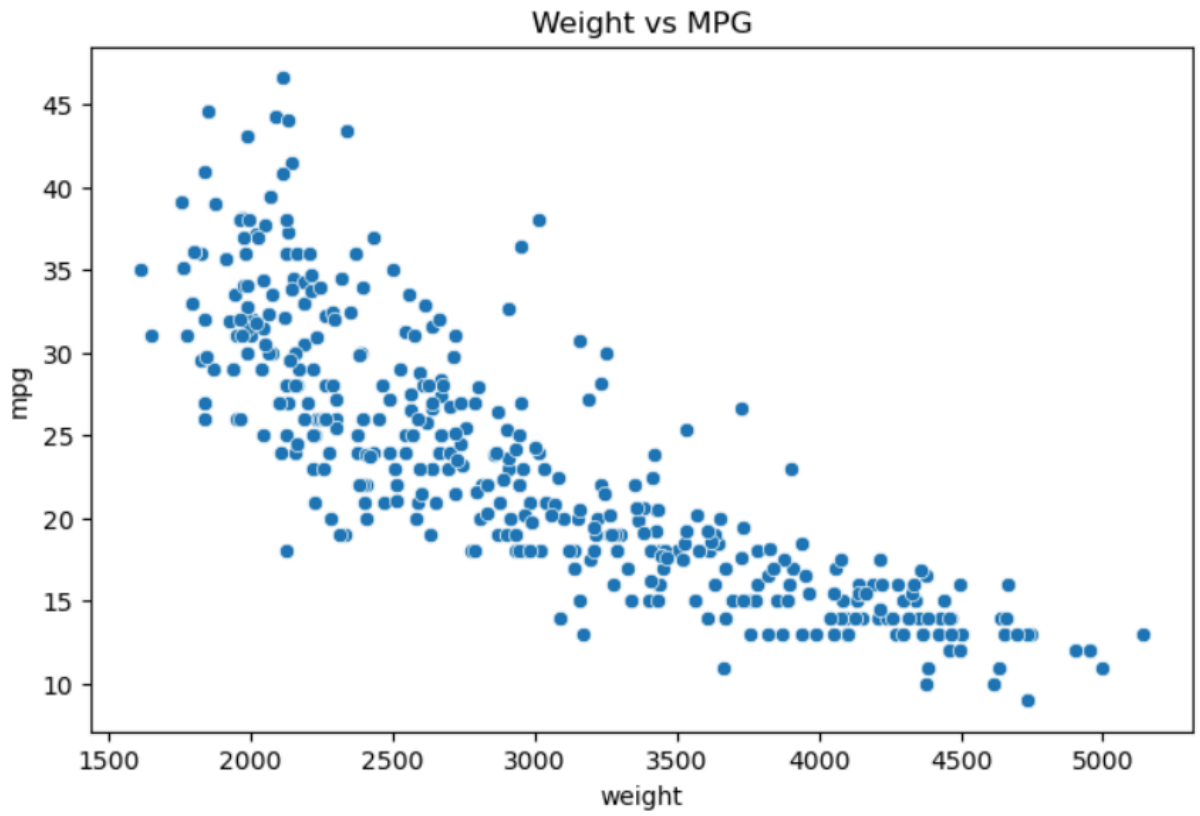

Figure 3. Weight vs. MPG

Scatterplot showing that heavier vehicles tend to have significantly lower MPG. Weight appears to be one of the strongest predictors.

A correlation heatmap confirms these relationships:

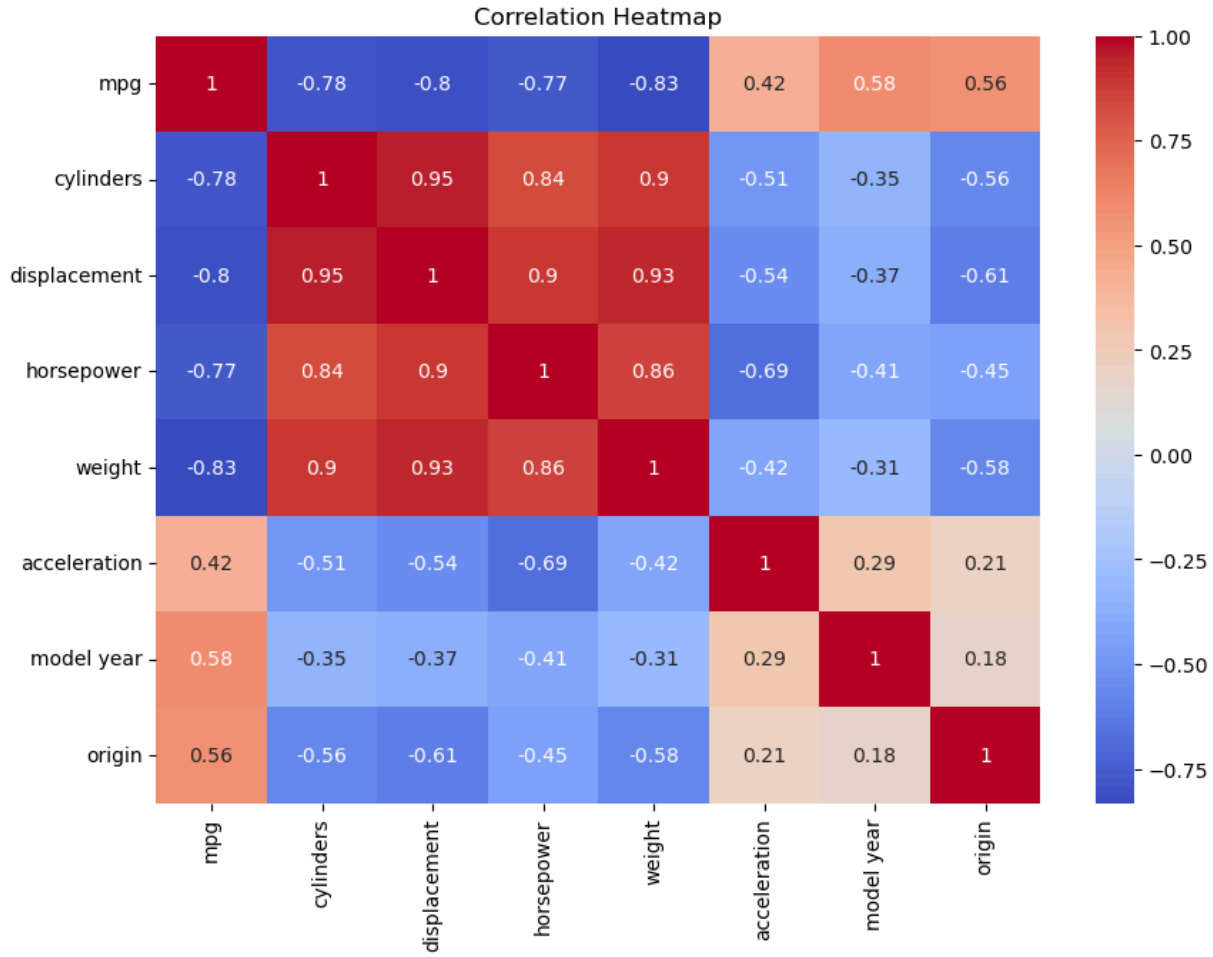

Figure 4. Correlation Heatmap

Visualizes pairwise correlations between all features. Strong multicollinearity is seen among cylinders, displacement, horsepower, and weight.

Key insights:

- Weight, horsepower, displacement, and cylinders are strongly negatively correlated with MPG
- Model year and origin have positive correlations (newer/lighter cars are more efficient)
- Horsepower contains missing values → preprocessing needed

Preprocessing

To prepare the data:

1. Removed rows missing MPG
2. Imputed missing horsepower values using the median
3. Scaled all numerical features for gradient descent
4. Applied one hot encoding to categorical features
5. Performed an 80/20 train test split

These steps improve algorithm stability and accuracy.

Methods

Ordinary Least Squares (OLS)

OLS solves for the weight vector analytically:

$$w = (X^T X)^{-1} X^T y$$

Advantages:

- Closed-form solution
- Fast for small to medium-sized datasets

Limitations:

- Requires matrix inversion
- Sensitive to multicollinearity
- Can suffer from numerical instability

Least Mean Squares (LMS)

LMS uses gradient descent:

$$w := w - \eta \, X^T (Xw - y)$$

where η is the learning rate.

Advantages:

- Does not require matrix inversion
- Handles poorly conditioned matrices better
- Works well with scaled data

Limitations:

- Requires tuning learning rate
- Needs many iterations

The loss curve below shows smooth convergence over 2,000 iterations:

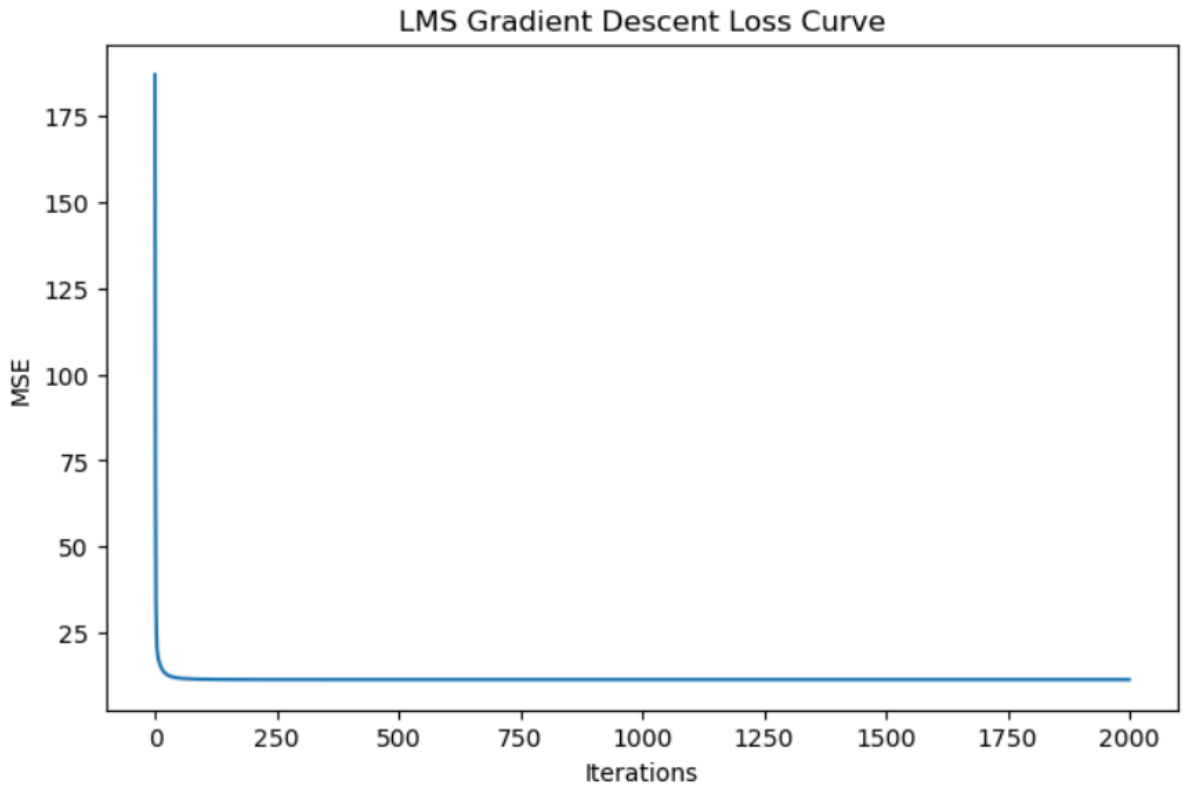

Figure 5. LMS Gradient Descent Loss Curve

Shows the LMS model's mean squared error decreasing smoothly over 2,000 iterations, indicating stable convergence with the chosen learning rate.

Results

Both models were evaluated using **Root Mean Squared Error (RMSE)** on train and test sets.

Model	Train RMSE	Test RMSE	Notes

OLS	22.99	20.65	Closed-form baseline
LMS	**3.37**	**2.89**	Significantly better performance

Interpretation

The LMS model greatly outperformed OLS. Although OLS is theoretically optimal for linear regression, in practice:

- The matrix XTXX^T XXTX was poorly conditioned because many features are highly correlated
- Matrix inversion amplified numerical errors
- LMS benefited from standardized features and iterative optimization

The LMS loss curve demonstrated stable descent toward an optimal solution, confirming that the chosen learning rate (0.001) and preprocessing pipeline were appropriate.

Analysis

The strong performance of LMS highlights that iterative methods can outperform analytical solutions when feature correlations cause instability in the OLS formula. Given the automotive dataset's high multicollinearity (e.g., displacement, horsepower, cylinders, and weight all correlate heavily), the OLS inverse matrix becomes sensitive to small floating-point errors.

LMS avoids matrix inversion and instead finds a solution through progressive updates, which can generalize better to unseen data.

In addition, the visualizations support the model's results:

- Scatterplots show clear linear downward trends for horsepower and weight
- The heatmap reveals strong multicollinearity among engine-related features
- This implies that regularized models (ridge/lasso) might perform even better

Overall, the project shows that even simple linear models require careful consideration of data conditioning and preprocessing.

Conclusion

This project demonstrated how machine learning can predict vehicle fuel efficiency using real-world automotive data. Through experimentation with OLS and LMS, I learned:

- Preprocessing (scaling, encoding, imputing) dramatically affects model behavior
- Gradient descent–based LMS can outperform analytical OLS in practical scenarios
- Strong multicollinearity impacts OLS performance
- Visualization is essential for understanding feature relationships

The greatest challenges involved learning rate selection, managing correlated features, and ensuring stable training.

Future improvements would include:

- Using ridge or lasso regression to handle multicollinearity
- Trying polynomial features or tree based models
- Incorporating vehicle aerodynamic data or dataset expansion

This project strengthened my understanding of regression, optimization, and model evaluation.

References

"Auto MPG Dataset." Kaggle, Version 1.0, 2024, www.kaggle.com/datasets/uciml/autompg-dataset.

Lee, Minwoo, and Hongfei Xue. "Gradient Descent and Least Mean Squares." *ITCS 3156*, University of North Carolina at Charlotte, 2025. Course Lecture.

Lee, Minwoo, and Hongfei Xue. "Linear Regression." *ITCS 3156*, University of North Carolina at Charlotte, 2025. Course Lecture.

"pandas." pandas.DataFrame.map, 2024, pandas.pydata.org/docs/reference/api/pandas.DataFrame.map.html.

Pedregosa, Fabian, et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–2830.

Virtanen, Pauli, et al. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." *Nature Methods*, vol. 17, 2020, pp. 261–272.

Walt, Stefan van der, et al. “The NumPy Array: A Structure for Efficient Numerical Computation.” *Computing in Science & Engineering*, vol. 13, no. 2, 2011, pp. 22–30.

Acknowledgement

I, Isaiah Alvarado-Ramirez, acknowledge that I used AI tools, specifically ChatGPT, as a supplementary resource during the completion of this project. AI was used in the following ways:

- Concept clarification: I asked AI to help me review definitions and ideas from class, such as how OLS works mathematically, how gradient descent updates weights, and why multicollinearity affects matrix inversion.
- Writing assistance: I used AI to help me revise sections of my report for clarity, fix grammar, and improve organization. I rewrote the text myself afterward so the explanations matched my own understanding and style.
- Debugging guidance: When I ran into issues with my Jupyter Notebook (such as formatting equations, handling missing values, or structuring plots), I asked AI for suggestions on how to correct them. I implemented all changes manually and verified the results myself.
- Project structure suggestions: I used AI for advice on how to organize my GitHub repository and how to present figures within the report.

AI did not generate the final code, train the models, analyze the results, interpret the RMSE values, or write the conclusions. All source code, data exploration, model training, analysis, and final decisions in this project were done by me.

Source Code

GitHub Repository: *https://github.com/ialvara1/ITCS-3156-Final/tree/main*